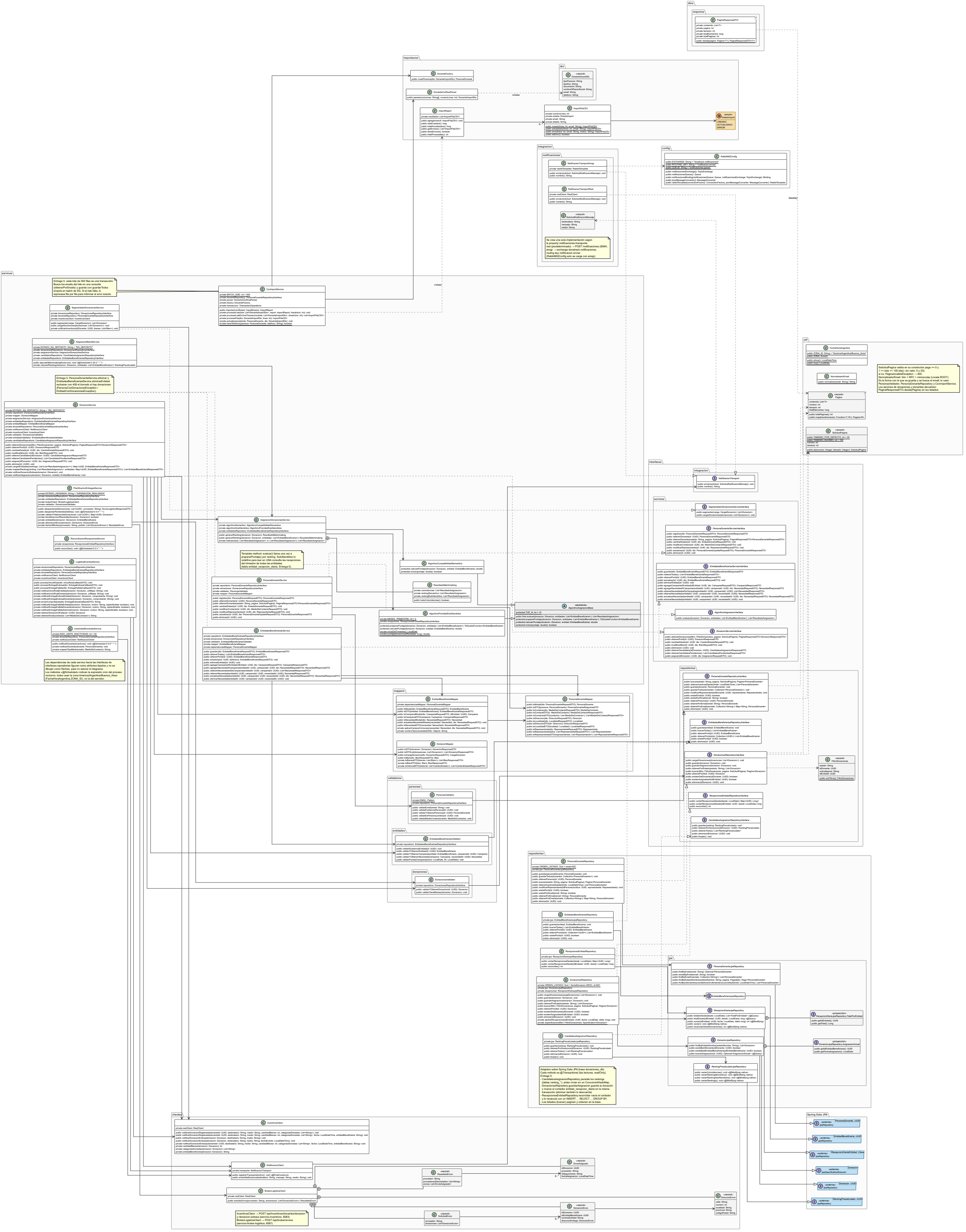

dtos
response
PaginaResponseDTO<T>
importacion
dto
DonanteImportDto
DonanteFactory
DonanteCsvRowParser
ImportReport
ImportFilaCSV
EstadoImport
integracion
notificaciones
RabbitMQConfig
NotificacionTransportAmqp
NotificacionTransportRest
SolicitudNotificacionMensaje
Se crea una sola implementación según
la property notificaciones.transporte:
rest (predeterminado) → POST /notificaciones (8084)
amqp → exchange donatrack.notificaciones,
routing key notificacion.enviar
(RabbitMQConfig solo se carga con amqp)
config
desde()
services
CsvImportService
SegmentadorDonacionesService
AsignacionBatchService
Entrega 5: cada lote de 500 filas es una transacción. Busca los emails del lote en una consulta (obtenerPorEmails) y guarda con guardarTodos (inserts en batch de 50). Si el lote falla, lo reprocesa fila por fila para informar el error exacto.
Entrega 5: PersonaDonanteService eliminar y EntidadesBeneficiariasService.eliminarEntidad rechazan con 409 el borrado si hay donaciones (PersonaConDonacionesExcepton / EntidadConDonacionesExcepton)
DonacionService
PlanificacionEntregasService
ReconciliacionRecepcionesService
LogisticaEventosService
InactividadDonantesService
Las dependencias de cada service hacia las interfaces de interfaces.repositorios figuran como atributos tipados y no se dibujan como flechas, para no saturar el diagrama.
Los métodos «@Scheduled» indican la expresión cron del proceso nocturno; todos usan la zona America/Argentina/Buenos_Aires (FechaHoraArgentina.ZONA_ID), no la del servidor.
util
FechaHoraArgentina
NormalizadorEmail
Pagina<T>
SolicitudPagina
SolicitudPagina valida en su constructor page >= 0 y
1 <= size <= 100 (de(): sin valor, 0 y 20);
si no, PaginaInvalidaException → 400.
NormalizadorEmail: trim + NFC + minúsculas (Locale.ROOT).
Es la forma con la que se guarda y se busca el email, lo usan PersonasValidator, PersonaDonanteRepository y CsvImportService.
Los servicios de donaciones y donantes devuelven PaginaResponseDTO.desde(Pagina) en los listados.
interfaces
integracion
NotificacionTransport
services
SegmentadorDonacionesServiceInterface
PersonaDonanteServiceInterface
EntidadesBeneficiariasServiceInterface
AlgoritmoAsignacionInterface
DonacionServiceInterface
repositories
PersonaDonanteRepositoryInterface
EntidadesBeneficiariasRepositoryInterface
DonacionesRepositoryInterface
RecepcionesEntidadRepositoryInterface
CandidatosAsignacionRepositoryInterface
mappers
EntidadBeneficiariaMapper
PersonaDonanteMapper
DonacionMapper
validations
personas
PersonasValidator
entidades
EntidadesBeneficiariasValidator
donaciones
DonacionesValidator
repositories
PersonaDonanteRepository
EntidadesBeneficiariasRepository
RecepcionesEntidadRepository
DonacionesRepository
CandidatosAsignacionRepository
jpa
PersonaDonanteJpaRepository
EntidadBeneficiariaJpaRepository
RecepcionDiariaJpaRepository
DonacionJpaRepository
RankingPrecalculadoJpaRepository
Adapters sobre Spring Data JPA (base donaciones_db).
Cada método es @Transactional (las lecturas, readOnly).
Entrega 5:
- CandidatosAsignacionRepository persiste los rankings
(tablas ranking_*); antes vivían en un ConcurrentHashMap.
- DonacionesRepository guardaAsignacion guarda la donación
y mueve el contador entidad_recepcion_diaria en la misma
transacción (eliminar también lo descuenta).
- RecepcionesEntidadRepository.reconciliar vacía el contador
y lo recalcula con un INSERT ... SELECT ... GROUP BY.
- Los listados (buscar) paginan y ordenan en la base.
Spring Data JPA
clientes
IncentivosClient
NotificacionClient
BrokerLogisticaClient
IncentivosClient → POST /api/incentivos/donantes/donacion y /donacion-exitosa (servicio-incentivos, 8083)
BrokerLogisticaClient → POST /api/broker/envios (servicio-broker-logistica, 8087)